Task 2 – Datasheet Literacy

Datasheet-Based Design Analysis: L298N Motor Driver

1. Datasheet Parameters and Their Influence on Circuit Design

The L298N is a dual full-bridge motor driver used to control the direction and speed of two DC motors.(in my case and posibly 4 also)

a) Motor supply voltage

4.5–13.5 V.. this is the voltage suplied to the motors and this helped me choose the battrey for it and also the motors too...

b) Output current rating

1.2 A continuous per channel, 3.2 A peak. And also the peak current of a n20 motor is 0.8 A which is very suitable .. to use the above driver ..

c) Logic input voltage

2.7–5.5 V, with a separate motor supply, VCC covers both 3.3 V and 5 V logic, so I wired it directly to my microcontroller's logic rail with no level shifter. VM and VCC are separate pins on the module, so the motors run from the battery while the logic stays on a clean regulated rail. The motor supply never feeds the microcontroller pins (main advantage ..

2. Condition That Could Damage the Component

Exceeding the 15 V absolute maximum on VM. Above this, the output MOSFETs and internal circuitry can break down, and the damage is permanent. In practice the risk comes from a wrong battery (for example 4S Li-ion at 16.8 V), voltage spikes from the motor, or reversed polarity. The breakout module has no reverse-polarity protection, so reversed battery leads would also destroy it.

To reduce this risk, the motors' current requirements should be checked before use, including their stall current. The driver should be operated within its electrical and thermal ratings. The motor supply voltage must also remain within the permitted range.

How my design avoid the problems are that i will first use a 3A fuse .. and also im using a 7.4 v Lipo battery and buck converter to make sure no wrong volatge go into the motor driver ... and also one very very main advantage is using a buck converter makes sure we get a steady supply og 6v to the driver .. so steady voltage to the motors so having more control on the car caz we are using PID

Task 3 – PID Control system

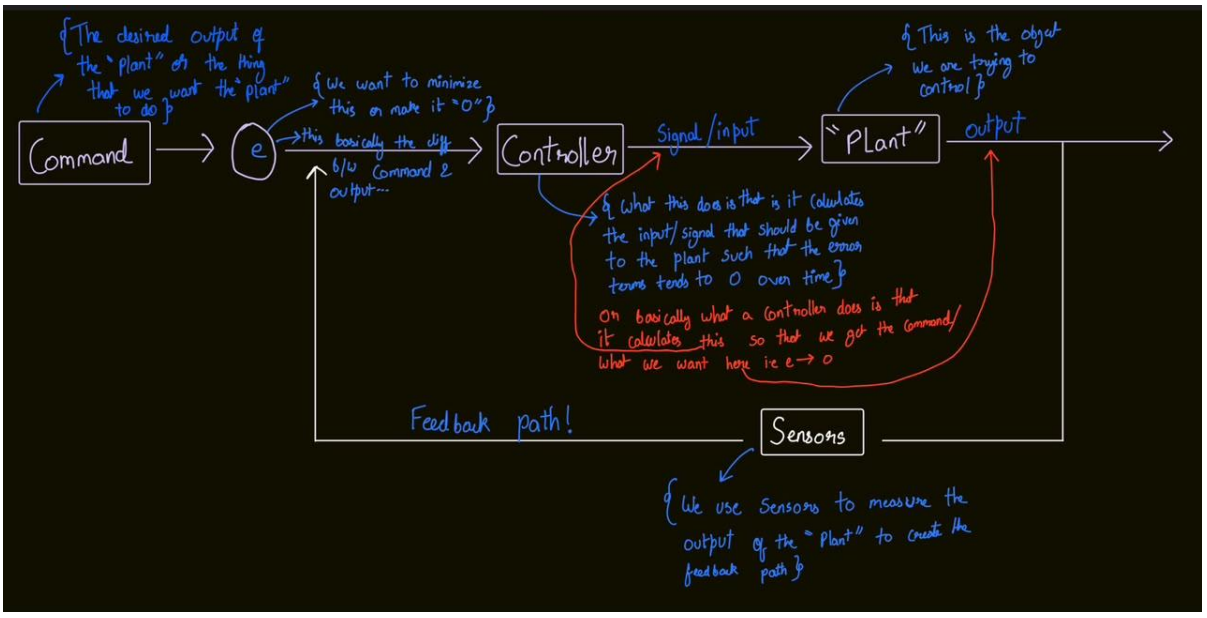
To understand what PID is we will first understand what a control system is …

Control System – It is a system that is designed to make our machine behave in our desired manner

- There are 2 type of control systems-
    - Open Loop Control System – Suppose a fan .. that runs for our desired time and switches off after the set time .. i.e basically a open loop control system does not change the inputs of the "plant" by/after checking its output.. I.e the fan does not check if it the desired result is achiefed or not .. it runs for the set time and turns off even if the room is still hot
    - Closed Loop Control System – This system contoinuously checks the output of the "plan" and keeps changing the input to the plant so that our desired result is achieved Say a smart ac which keeps measuring the room temperature and reduces the cooling as the room keeps getting cooler and finaly stops after reaching the desired temperature
    - So basically if the input is adjusted corresponding to the output to get the desired result it is called a closed loop
- We use sensors and a controller to change the input so that we get a desired output

- One Such controller in a closed loop control systems is PID

PID Controller

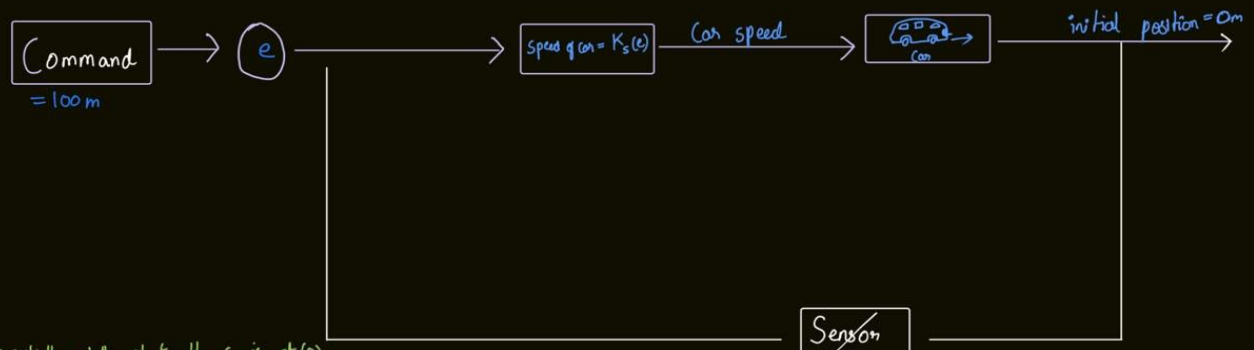
- Read the below so you can understand well (my handwritten stuff)
- 

6:32 PM Mon 26 Oct 32%

Linear algebra 1 | Untitled

Lets try to understand this by another example where the controller is a simple proportional controller..

Say we have a Car that has to travel 100 m ...



1) initially at the start the car is at (0)
So the error term (e) = 100 - 0 = 100 m

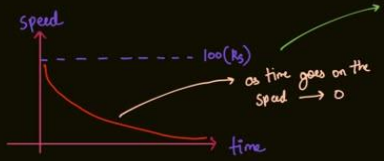
2) Since the e = 100 the controller tells the car to move at speed = K_s(100) Say K_s = (0.1) then S = 10 m/s

3) Say after some time the position of car moves closer to the 100 m so e term decreases & the speed also keeps decreasing

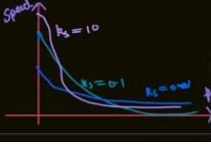
4) lets try to visualise this by some graphs..

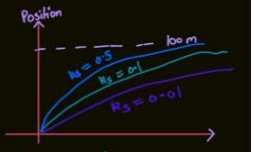
Speed = K_s (error)
Error = 100 - (position)





We can basically change the K_s to change how quick / slow we need to go the 100 m...





6:32 PM Mon 26 Oct 32%

Linear algebra 1 | Untitled

Now Why PID & why not just proportional controller..

→ So we saw that the proportional controller could do the task above & now lets discuss PID..

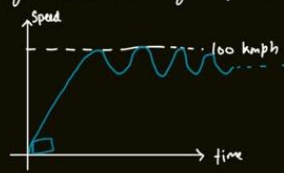
→ Suppose the example of a Cruise control car... & here
① Command = 100 km/h speed
② input given to the motors is say in RPM
③ Output is the speed of the car...
④ Sensor is the speedometer...

Case - I (Just proportional controller)

→ here say the car was at rest, the errors here will be max & the rpm of the motors will also be high...

→ So as the car speed gets closer and closer to 100 the errors gets closer to 0 & then rpm of the motor gets to 0 & so does the speed of the car..

→ now either of 2 thing happen... Say it reaches the 100 kmph, now the e becomes 0 so the, controller makes rpm 0 & the car starts to slow & e again starts increasing & again reaches 100 ... So on.....



→ So what happens is it reaches 100 but does not stay at that speed for long time...

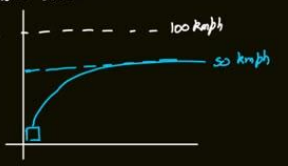
→ it can also so happen that it never reaches 100 kmph

Say 1000 rpm = for the car to cruise at 50 kmph..

→ Say K_p = 200 then
rpm = 20 (e)

initially e = 100 so rpm increases & so does speed & say like this speed keeps on increases & e = 50 kmph ie car reaches 50 kmph & at that speed
rpm = 20(50) = 1000.. but 1000 rpm corresponds exactly to maintain 50 kmph ie it comes to 50 kmph & stays & can never increase...

→ this is called the steady state errors..



Case - II (P + I)

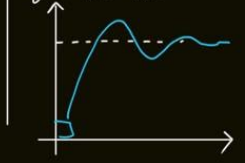
→ now to try solve the issue we add a integral term..

initially rpm = $K_p(e)$

now rpm = $K_p(e) + K_i \int e\,dt$

now what this does is it keeps adding all the error terms..

→ Consider the same example but now with the integral term...
So say the car reaches speed = 100 but this there the motors dont stop cos the integral term is not 0 .. & so it overshoots .. & as it overshoots the e becomes -ve & integral also starts decreasing ... So it goes like this

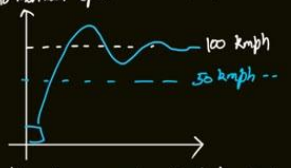


So in P+I controller compared to the P controller there are say less "jerks" and reaches desired value more controlled manner.... (which is highly important for line following robots..)

6:32 PM Mon 26 Oct — 32%

Linear algebra 1 — Untitled

→ Seems good right -- but there is still a tiny problem --- if you remember the steady error will also be solved because here even though $k_p(e) = 0$ the rpm req to maintain a speed but integral term is still +ve -- so total rpm > rpm req to maintain speed --- here it will no longer be stuck at a speed to ....



→ basically if you visualise before coming constant speed = 100 km/h the speed of the car overshoots --- & the value of e becomes 0 & the integral term reducing ---

→ even though it still comes to const speed = 100 km/h but that overshoot may not be desirable --

→ now how do we fix this ?

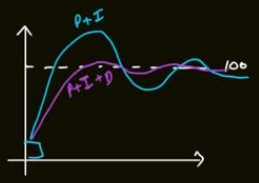
Case - III (P+I+D)

→ now what we do is we add a derivative term too so

$$rpm = k_p e + k_i \int e\,dt + k_d \frac{de}{dt} \rightarrow P+I+D$$

→ what this term does is it measures the change in e, say we are approaching speed = 100 so the e decreases & the $\frac{de}{dt} < 0$ so it tries to reduce the rpm --

→ So basically what the D term does is, it exactly reverses the effect of the integral ; i.e to compensate the overshoot the derivative creates & undershoot so that the overall rpm comes to 100 more smoothly basically!



67% | 4 of 5

Task 7 — Wireless Sensor Diagnostics

Protocol selection

- What we will do is compere all the 3 and then from the table we see what benifits us best in our case
-

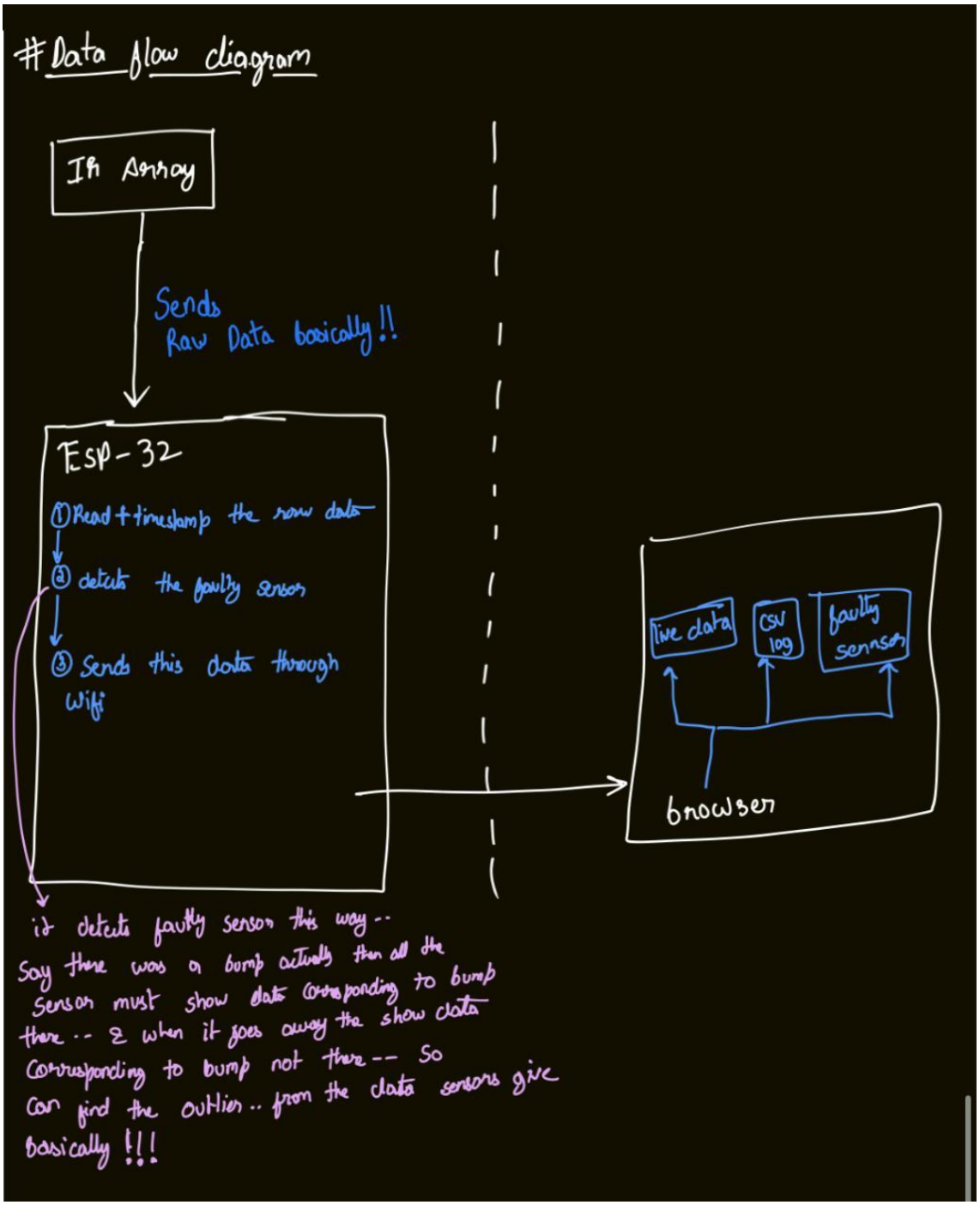
Criterion	Wi-Fi	Classic BT (SPP)	BLE
Latency	Low, but jitter is not so even basically	Low and steady	Low, but small packets and fixed intervals
Power	Highest, noisy bursts	Moderate	Lowest
Ease	Medium (needs AP, IP, client)	**Easiest**	Hardest (GATT setup)
Data rate vs. need	Far more than needed	**Enough** (~3 KB/s needed)	Enough, but must fragment data
Client	Browser or custom app	**Any serial terminal**	BLE app

- So what we will use is the WI-FI
    - **1. Latency**
        - Sensor glitches from bumps last only a few milliseconds, so I need data to arrive continuously and at a steady pace.
        - In Wifi mode, the debugger's phone or PC is the only device on the network. There is no shared traffic and no router in the path, so delay is low (roughly 5-20 ms) and steady.
        - We can apparently use a WebSocket so that as soon as the sensors get a reading we can ask the esp to push it to be on the web page ..
        - Classic Bluetooth SPP is similar in latency, so it has no real advantage here.
        - BLE sends small packets at fixed intervals, so data arrives in bursts and a short glitch can be smeared or missed.
    - **2. Power**
        - BLE uses the least power, but this is not a deciding factor. The motors draw far more than any BLE.
        - Wi-Fi draws the most radio current, in bursts of a few hundred mA. This is well within the 5 V rail budget (0.37 A maximum upar calculate kia he), and the motors run on their own 6 V buck, so the bursts will not cause a meaningful voltage drop i hbelieve. A capacitor near the ESP32's 5 V input should cover any small baxodi in voltage i believe.
        - So power ko dekh ke kisi cheez ko rule out nahi karenge coz almos saari hi similar he
    - **3. Ease of implementation**
        - The ESP32 hosts a small web page and a WebSocket server. Any phone or PC with a browser can connect, so no app has to be installed.
        - Writing the page (a live plot of the 8 sensors, with the faulty channel highlighted) is a small task, and the same page also provides the logging (a CSV download button).

- It works on Android, iOS, Windows and Linux alike. Classic Bluetooth SPP does not work on iPhones.
- Classic Bluetooth is simple for raw text, but it still needs a serial terminal app or a Python script to plot the data. That is about the same effort as the web page and gives a worse display.
- BLE needs GATT services and characteristics and a BLE app, which is the hardest of the three

- **4.Utkarsh Reasons**
  - Meri utkarsh me gaddi wifi baaar baar disconnect ho rhi thi .. ab baat ego pe aagyi he meri so issbar same problem aayigi toh me fix karke hi rahunga bc 😤😤🥀🥀

System architecture



Task 8 — Maze-Solving Algorithms

Comparison